

MANUAL TÉCNICO - FIUSAT Rating

Universidad de San Carlos de Guatemala

Facultad de Ingeniería - Escuela de Ciencias y Sistemas

Prácticas Iniciales 2026

Por Luis Zepeda-202403162

1. INTRODUCCIÓN

1.1 Propósito del Documento

Este manual técnico describe la implementación completa de la aplicación FIUSAT Rating, incluyendo arquitectura, tecnologías, estructura del código, endpoints de la API REST, y procedimientos de instalación y configuración.

1.2 Alcance

Este documento está dirigido a:

 Desarrolladores que necesitan entender y mantener el sistema

 Administradores que realizarán la instalación y mantenimiento

 Evaluadores que verificarán el funcionamiento técnico

 Estudiantes que deseen entender la implementación

1.3 Descripción del Sistema

FIUSAT Rating es una aplicación web full-stack que permite a los estudiantes de la Facultad de Ingeniería de la USAC:

Registrar y autenticar usuarios

Crear, editar y eliminar publicaciones sobre cursos y catedráticos

Comentar y interactuar con publicaciones

Gestionar perfiles y cursos aprobados

Buscar y filtrar información

Eliminar cuentas de forma segura

2. ARQUITECTURA DEL SISTEMA

2.1 Arquitectura Cliente-Servidor



2.2 Flujo de Datos

Paso	Descripción	Tecnología
1	Usuario interactúa con el frontend	React Components
2	Frontend envía petición HTTP	Axios
3	Backend recibe y valida la petición	Express + Middleware
4	Backend consulta la base de datos	MySQL2/Promise
5	Base de datos retorna los datos	MySQL 8.0
6	Backend envía respuesta JSON	Express Response
7	Frontend muestra los datos al usuario	React State
8	Notificación de confirmación	Notification Component

3. TECNOLOGÍAS UTILIZADAS

3.1 Backend

Tecnología	Versión	Propósito	Documentación	↓
Node.js	18.x o superior	Entorno de ejecución	https://nodejs.org	
Express	4.18.x	Framework web	https://expressjs.com	
MySQL2	3.6.x	Conexión a BD	https://www.npmjs.com/package/mysql2	
Bcryptjs	2.4.x	Encriptación	https://www.npmjs.com/package/bcryptjs	
JSON Web Token	9.0.x	Autenticación	https://www.npmjs.com/package/jsonwebtoken	
Dotenv	16.3.x	Variables de entorno	https://www.npmjs.com/package/dotenv	
Cors	2.8.x	CORS	https://www.npmjs.com/package/cors	

3.2 Frontend

Tecnología	Versión	Propósito	Documentación
React	18.2.x	Framework de UI	https://react.dev
React Router DOM	6.15.x	Navegación	https://reactrouter.com
Axios	1.5.x	Peticiones HTTP	https://axios-http.com

3.3 Base de Datos

Tecnología	Versión	Propósito
MySQL Server	8.0+	Gestor de BD relacional
MySQL Workbench	8.0+	Administración visual

3.4 Herramientas de Desarrollo

Herramienta	Propósito
Visual Studio Code	Editor de código
Postman	Pruebas de API
Git & GitHub	Control de versiones
npm	Gestor de paquetes
Nodemon	Auto-reinicio del servidor

4. ESTRUCTURA DEL PROYECTO

4.1 Árbol de Directorios Completo

```
fiusat_rating/
|
|— backend/
|  |— node_modules/
|  |— routes/
|  |  |— auth.js      # Autenticación (login, registro, reset)
|  |  |— usuarios.js   # CRUD usuarios y cursos aprobados
|  |  |— publicaciones.js # CRUD publicaciones
|  |  |— comentarios.js # CRUD comentarios
|  |  |— cursos.js     # CRUD cursos
|  |  |— catedraticos.js # CRUD catedráticos
|  |— .env            # Variables de entorno
|  |— db.js           # Conexión a MySQL (pool)
|  |— index.js        # Punto de entrada del servidor
```

```
| | └─ package.json      # Dependencias del backend
| | └─ package-lock.json # Versiones exactas
|
| └─ frontend/
| | └─ node_modules/
| | └─ public/
| |   └─ index.html      # HTML base
| | └─ src/
| |   └─ components/
| |     └─ Login.jsx      # Componente de login
| |     └─ Login.css
| |     └─ Register.jsx   # Componente de registro
| |     └─ Register.css
| |     └─ Feed.jsx       # Feed de publicaciones
| |     └─ Feed.css
| |     └─ PublicacionCard.jsx # Tarjeta de publicación
| |     └─ PublicacionCard.css
| |     └─ CrearPublicacion.jsx # Crear publicación
| |     └─ CrearPublicacion.css
| |     └─ Perfil.jsx     # Perfil de usuario
| |     └─ Perfil.css
| |     └─ CursosAprobados.jsx # Cursos aprobados
| |     └─ CursosAprobados.css
| |     └─ Navbar.jsx     # Barra de navegación
| |     └─ Navbar.css
| |     └─ Notification.jsx # Sistema de notificaciones
| |     └─ Notification.css
```

```

| | | └─ services/
| | |   └─ api.js      # Configuración de Axios + Interceptors
| | | └─ App.js        # Componente principal y rutas
| | | └─ index.js      # Punto de entrada de React
| | | └─ index.css     # Estilos globales
| | |   └─ App.css     # Estilos de App
| | └─ package.json    # Dependencias del frontend
|   └─ package-lock.json
|
└─ database.sql        # Script de creación de BD
└─ .gitignore          # Archivos a ignorar en Git
└─ README.md           # Documentación del repositorio
└─ MANUAL_USUARIO.md   # Manual de usuario
└─ MANUAL_TECNICO.md   # Este archivo

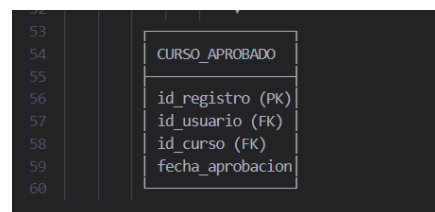
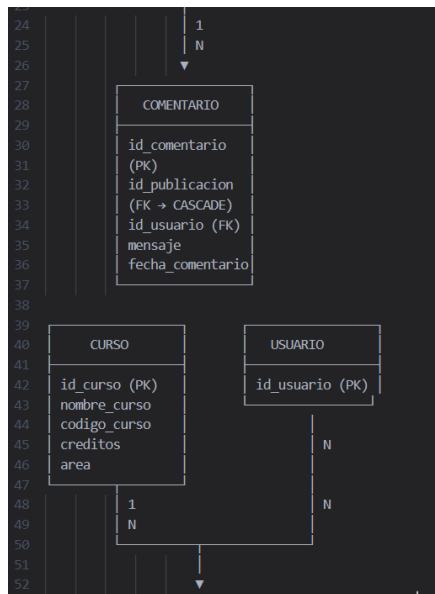
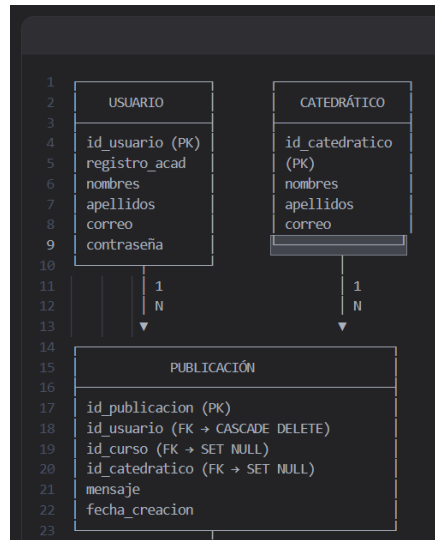
```

4.2 Descripción de Archivos Principales

Archivo	Descripción	Tamaño Aprox. 
<code>backend/index.js</code>	Configura Express, middleware y monta las rutas	~50 líneas
<code>backend/db.js</code>	Establece conexión con MySQL usando pool	~15 líneas
<code>backend/routes/auth.js</code>	Maneja registro, login y reset de contraseña	~150 líneas
<code>backend/routes/usuarios.js</code>	CRUD de usuarios y cursos aprobados	~180 líneas
<code>backend/routes/publicaciones.js</code>	CRUD completo de publicaciones	~200 líneas
<code>backend/routes/comentarios.js</code>	CRUD completo de comentarios	~150 líneas
<code>frontend/src/App.js</code>	Define las rutas de la aplicación React	~40 líneas
<code>frontend/src/services/api.js</code>	Configura Axios con interceptors para JWT	~30 líneas
<code>frontend/src/components/Notification.js</code>	Componente de notificaciones	~60 líneas

5. BASE DE DATOS

5.1 Diagrama Entidad-Relación



5.2 Estructura de Tablas

Tabla: USUARIO

sql

```
1 CREATE TABLE usuario (  
2     id_usuario INT AUTO_INCREMENT PRIMARY KEY,  
3     registro_academico VARCHAR(20) NOT NULL UNIQUE,  
4     nombres VARCHAR(100) NOT NULL,  
5     apellidos VARCHAR(100) NOT NULL,  
6     correo VARCHAR(100) NOT NULL UNIQUE,  
7     contraseña VARCHAR(255) NOT NULL,  
8     fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Tabla: CATEDRÁTICO

sql

```
1 CREATE TABLE catedratico (  
2     id_catedratico INT AUTO_INCREMENT PRIMARY KEY,  
3     nombres VARCHAR(100) NOT NULL,  
4     apellidos VARCHAR(100) NOT NULL,  
5     correo VARCHAR(100)  
6 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Tabla: CURSO

sql

```
1 CREATE TABLE curso (  
2     id_curso INT AUTO_INCREMENT PRIMARY KEY,  
3     nombre_curso VARCHAR(100) NOT NULL,  
4     codigo_curso VARCHAR(20),  
5     creditos INT,  
6     area VARCHAR(50) DEFAULT 'Sistemas'  
7 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Tabla: PUBLICACIÓN

sql

```
1 CREATE TABLE publicacion (  
2     id_publicacion INT AUTO_INCREMENT PRIMARY KEY,  
3     id_usuario INT NOT NULL,  
4     id_curso INT,  
5     id_catedratico INT,  
6     mensaje TEXT NOT NULL,  
7     fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
8     FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE,  
9     FOREIGN KEY (id_curso) REFERENCES curso(id_curso) ON DELETE SET NULL,  
10    FOREIGN KEY (id_catedratico) REFERENCES catedratico(id_catedratico) ON DELETE SET NULL  
11 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Tabla: COMENTARIO

sql

```
1 CREATE TABLE comentario (  
2     id_comentario INT AUTO_INCREMENT PRIMARY KEY,  
3     id_publicacion INT NOT NULL,  
4     id_usuario INT NOT NULL,  
5     mensaje TEXT NOT NULL,  
6     fecha_comentario TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
7     FOREIGN KEY (id_publicacion) REFERENCES publicacion(id_publicacion) ON DELETE CASCADE,  
8     FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE  
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Tabla: CURSO_APROBADO

sql

```
1 CREATE TABLE curso_aprobado (  
2     id_registro INT AUTO_INCREMENT PRIMARY KEY,  
3     id_usuario INT NOT NULL,  
4     id_curso INT NOT NULL,  
5     fecha_aprobacion DATE,  
6     UNIQUE KEY unique_usuario_curso (id_usuario, id_curso),  
7     FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario) ON DELETE CASCADE,  
8     FOREIGN KEY (id_curso) REFERENCES curso(id_curso) ON DELETE CASCADE  
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6. API REST - ENDPOINTS

6.1 Convenciones

Elemento	Convención	Ejemplo
URL Base	<code>http://localhost:3001/api</code>	-
Método HTTP	GET, POST, PUT, DELETE	-
Formato de datos	JSON	<code>{"key": "value"}</code>
Códigos de estado	200, 201, 400, 401, 404, 500	-
Autenticación	Bearer Token en Header	<code>Authorization: Bearer <token></code>
Respuesta éxito	<code>{ message: "...", data: {...} }</code>	-
Respuesta error	<code>{ error: "...", message: "..."} </code>	-

6.2 Endpoints de Autenticación (/api/auth)

POST /api/auth/register

Registra un nuevo usuario en el sistema.

Request:

```

http
1  POST http://localhost:3001/api/auth/register
2  Content-Type: application/json
3
4  {
5      "registro_academico": "202012345",
6      "nombres": "Juan Carlos",
7      "apellidos": "Pérez López",
8      "correo": "juan@test.com",
9      "contraseña": "123456"
10 }
```

Response (201 - Éxito):

json



```
1 {  
2   "message": "Usuario registrado con éxito. Ahora puedes iniciar sesión."  
3 }
```

Response (400 - Error):

json



```
1 {  
2   "message": "El usuario ya existe. El correo o registro académico ya están registrados."  
3 }
```

Validaciones:

- Todos los campos son requeridos
- registro_academico: 8-9 dígitos numéricos
- contraseña: mínimo 6 caracteres
- correo: formato email válido
- correo y registro_academico deben ser únicos

POST /api/auth/login

Autentica un usuario y devuelve un token JWT.

Request:

http

```
1 POST http://localhost:3001/api/auth/login  
2 Content-Type: application/json  
3  
4 {  
5   "correo": "juan@test.com",  
6   "contraseña": "123456"  
7 }
```

Response (200 - Éxito):

json

```
1 {
2   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
3   "user": {
4     "id": 1,
5     "nombres": "Juan Carlos",
6     "apellidos": "Pérez López",
7     "registro": "202012345",
8     "correo": "juan@test.com"
9   }
10 }
```

Response (401 - Error):

json

```
1 {
2   "message": "Credenciales incorrectas"
3 }
```

Proceso:

1. Busca usuario por correo
2. Verifica contraseña con `bcrypt.compare()`
3. Genera token JWT con payload { id, registro, nombres }
4. Token expira en 24 horas

PUT `/api/auth/reset-password`

Restablece la contraseña de un usuario.

Request:

http

```
1 PUT http://localhost:3001/api/auth/reset-password
2 Content-Type: application/json
3
4 {
5   "registro_academico": "202012345",
6   "correo": "juan@test.com",
7   "nueva_contraseña": "nueva123"
8 }
```

6.3 Endpoints de Usuarios (/api/usuarios)

GET /api/usuarios/:id

Obtiene la información de un usuario por ID.

Request:

```
http

1 GET http://localhost:3001/api/usuarios/1

Response (200 - Éxito):

json

1 {
2   "id_usuario": 1,
3   "registro_academico": "202012345",
4   "nombres": "Juan Carlos",
5   "apellidos": "Pérez López",
6   "correo": "juan@test.com"
7 }

Response (404 - Error):

json

1 {
2   "message": "Usuario no encontrado"
3 }
```

GET /api/usuarios/registro/:registro

Busca un usuario por registro académico.

Request:

http

```
1 GET http://localhost:3001/api/usuarios/registro/202012345
```

Response (200 - Éxito):

json

```
1 {
2   "id_usuario": 1,
3   "registro_academico": "202012345",
4   "nombres": "Juan Carlos",
5   "apellidos": "Pérez López",
6   "correo": "juan@test.com"
7 }
```

PUT /api/usuarios/:id

Actualiza la información de un usuario.

Request:

http

```
1 PUT http://localhost:3001/api/usuarios/1
2 Content-Type: application/json
3
4 {
5   "nombres": "Juan Carlos",
6   "apellidos": "Pérez López",
7   "correo": "nuevo@test.com"
8 }
```

Response (200 - Éxito):

json

```
1 {
2   "message": "Perfil actualizado correctamente"
3 }
```

Validaciones:

nombres, apellidos y correo son requeridos

correo debe ser formato email válido

DELETE /api/usuarios/:id ☆

Elimina un usuario y todos sus datos asociados (CASCADE).

Request:

http

```
1 DELETE http://localhost:3001/api/usuarios/1
2 Content-Type: application/json
3
4 {
5   "id_usuario": 1
6 }
```

Response (200 - Éxito):

json

```
1 {
2   "message": "Cuenta eliminada correctamente"
3 }
```

Response (404 - Error):

json

```
1 {
2   "message": "Usuario no encontrado"
3 }
```

Eliminación en Cascada:

- ☒ Se eliminan todas las publicaciones del usuario
- ☒ Se eliminan todos los comentarios del usuario
- ☒ Se eliminan los cursos aprobados del usuario

GET /api/usuarios/:id/cursos

Obtiene los cursos aprobados de un usuario.

Request:

http

```
1 GET http://localhost:3001/api/usuarios/1/cursos
```

0

Response (200 - Éxito):

json

```
1  [
2    {
3      "id_curso": 1,
4      "nombre_curso": "Programación 1",
5      "codigo_curso": "CC101",
6      "creditos": 5,
7      "area": "Sistemas",
8      "fecha_aprobacion": "2026-03-15"
9    },
10   {
11     "id_curso": 2,
12     "nombre_curso": "Programación 2",
13     "codigo_curso": "CC201",
14     "creditos": 5,
15     "area": "Sistemas",
16     "fecha_aprobacion": "2026-03-16"
17   }
18 ]
```


6.4 Endpoints de Publicaciones (/api/publicaciones)

GET /api/publicaciones

Obtiene todas las publicaciones con filtros opcionales.

Request:

http

```
1 GET http://localhost:3001/api/publicaciones
2 GET http://localhost:3001/api/publicaciones?id_curso=1
3 GET http://localhost:3001/api/publicaciones?id_catedratico=1
```

Response (200 - Éxito):

json

```
1 [
2   {
3     "id_publicacion": 1,
4     "id_usuario": 1,
5     "id_curso": 1,
6     "id_catedratico": null,
7     "mensaje": "Excelente curso, muy recomendado",
8     "fecha_creacion": "2026-03-18T10:30:00.000Z",
9     "autor_nombre": "Juan Carlos",
10    "autor_apellido": "Pérez López",
11    "autor_registro": "202012345",
12    "nombre_curso": "Programación 1",
13    "catedratico_nombre": null,
14    "catedratico_apellido": null
15  }
16 ]
```

Query Parameters:

- id_curso (opcional): Filtrar por curso
- id_catedratico (opcional): Filtrar por catedrático

6.5 Endpoints de Comentarios (/api/comentarios)

GET /api/comentarios/publicacion/:id

Obtiene todos los comentarios de una publicación.

Request:

```
http

1 GET http://localhost:3001/api/comentarios/publicacion/1

Response (200 - Éxito):

json

1 [
2   {
3     "id_comentario": 1,
4     "id_publicacion": 1,
5     "id_usuario": 2,
6     "mensaje": "Totalmente de acuerdo",
7     "fecha_comentario": "2026-03-18T11:00:00.000Z",
8     "autor_nombre": "María",
9     "autor_apellido": "García"
10  }
11 ]
```

6.6 Endpoints de Cursos (/api/cursos)

GET /api/cursos

Obtiene todos los cursos disponibles.

Request:

Request:

http

```
1 GET http://localhost:3001/api/cursos
```

Response (200 - Éxito):

json

```
1 [
2   {
3     "id_curso": 1,
4     "nombre_curso": "Programación 1",
5     "codigo_curso": "CC101",
6     "creditos": 5,
7     "area": "Sistemas"
8   }
9 ]
```

6.7 Endpoints de Catedráticos (/api/catedraticos)

GET /api/catedraticos

Obtiene todos los catedráticos disponibles.

Request:

Request:

http

```
1 GET http://localhost:3001/api/catedraticos
```

Response (200 - Éxito):

json

```
1 [
2   {
3     "id_catedratico": 1,
4     "nombres": "Floriza",
5     "apellidos": "Avila",
6     "correo": "favila@usac.edu.gt"
7   }
8 ]
```

7. AUTENTICACIÓN Y SEGURIDAD

7.1 Encriptación de Contraseñas

Las contraseñas se encriptan usando **bcrypt** con un salt de 8 rondas:

javascript

```
1  const bcrypt = require('bcryptjs');
2
3  // En registro
4  const hashedPassword = await bcrypt.hash(contraseña, 8);
5
6  // En login
7  const validPassword = await bcrypt.compare(contraseña, hashedPassword);
```

Características:



Algoritmo: bcrypt



Salt rounds: 8



Almacenamiento: VARCHAR(255)



Costo computacional: Balanceado entre seguridad y rendimiento

7.2 JWT (JSON Web Tokens)

Los tokens JWT se utilizan para autenticar las peticiones del frontend:

javascript

```
1  const jwt = require('jsonwebtoken');
2
3  // Generar token (en login)
4  const token = jwt.sign(
5    {
6      id: user.id_usuario,
7      registro: user.registro_academico,
8      nombres: user.nombres
9    },
10   process.env.JWT_SECRET || 'secreto_key_fiusat_2026',
11   { expiresIn: '24h' }
12 );
13
14 // Verificar token (middleware opcional)
15 const decoded = jwt.verify(token, process.env.JWT_SECRET);
```

8. INSTALACIÓN Y CONFIGURACIÓN

8.1 Requisitos Previos

Software	Versión Mínima	Enlace	Estado
Node.js	18.x	https://nodejs.org	✓ Requerido
npm	9.x	Incluido con Node.js	✓ Requerido
MySQL Server	8.0+	https://mysql.com	✓ Requerido

8.2 Instalación del Backend

bash

```
1 # 1. Navegar al directorio del backend
2 cd backend
3
4 # 2. Instalar dependencias
5 npm install
6
7 # 3. Crear archivo .env
8 cp .env.example .env
9
10 # 4. Editar .env con las configuraciones
11 # DB_HOST=localhost
12 # DB_USER=root
13 # DB_PASSWORD=tu_contraseña
14 # DB_NAME=fiusat_rating
15 # PORT=3001
16 # JWT_SECRET=secreto_key_fiusat_2026_muy_seguro
17
18 # 5. Iniciar el servidor
19 npm start
20
21 # O con nodemon para auto-reiniciar
22 npm run dev
```

Verificación:

- 1  Servidor corriendo en <http://localhost:3001>
- 2  API lista para recibir peticiones

8.3 Instalación del Frontend

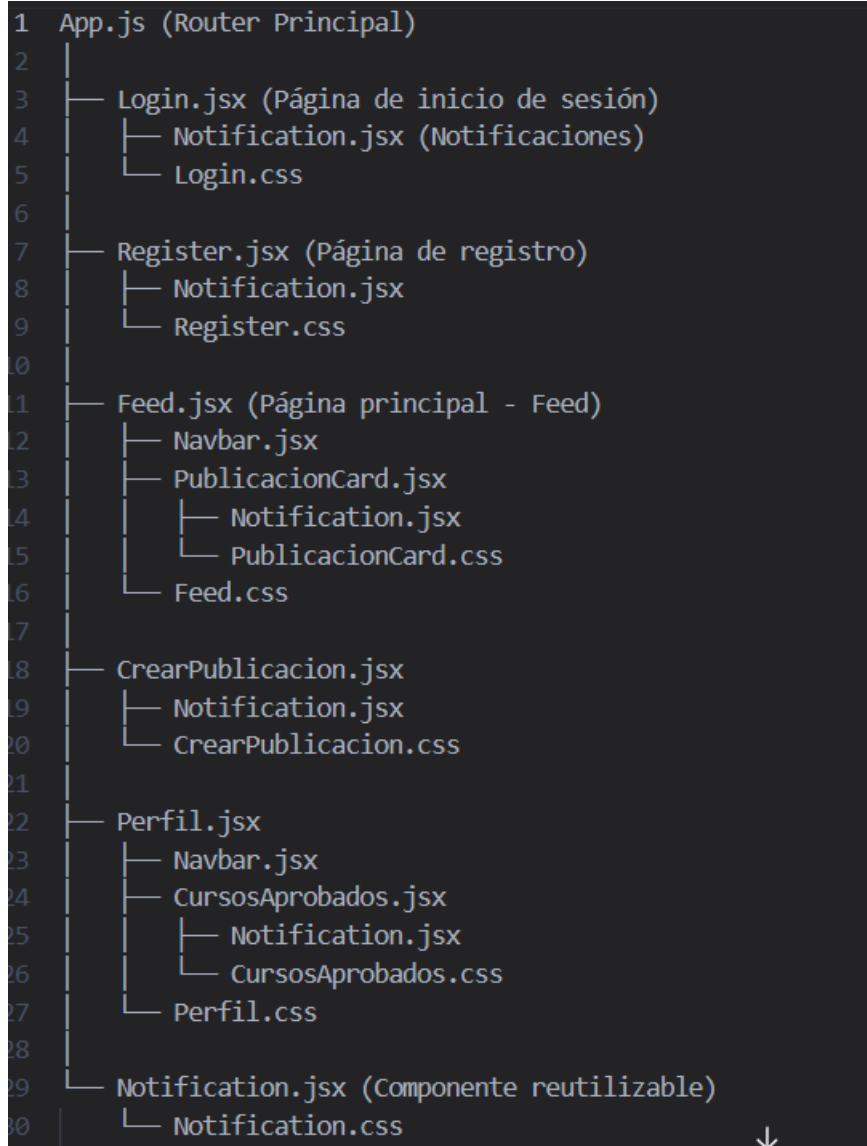
```
1 # 1. Navegar al directorio del frontend
2 cd frontend
3
4 # 2. Instalar dependencias
5 npm install
6
7 # 3. Iniciar el servidor de desarrollo
8 npm start
```

Verificación:

- El navegador se abre automáticamente en <http://localhost:3000>
- Si no se abre, acceder manualmente a <http://localhost:3000>

9. COMPONENTES DEL FRONTEND

9.1 Estructura de Componentes



9.2 Componente Notification.jsx

Propósito: Mostrar notificaciones amigables y personalizables.

Props:

```
javascript
1 {
2   message: string,    // Mensaje a mostrar
3   type: string,       // 'success' | 'error' | 'warning' | 'confirm' | 'info'
4   onConfirm: function, // Función al hacer clic en confirmar
5   onCancel: function,  // Función al hacer clic en cancelar
6   showCancel: boolean  // Mostrar botón de cancelar (default: false)
7 }
```

9.3 Componente PublicacionCard.jsx

Propósito: Mostrar una publicación con todas sus funcionalidades.

Props:

```
javascript

1 {
2   publicacion: object, // Datos de la publicación
3   onUpdate: function, // Callback al actualizar
4   onDelete: function   // Callback al eliminar
5 }
```

10. NOTIFICACIONES DEL SISTEMA

10.1 Características

Característica	Descripción
🎨 Diseño Moderno	Gradientes, sombras, bordes redondeados
🌟 Icono Grande	Icono de 56px con animación de pop
🌀 Anillo Animado	Círculo pulsante alrededor del icono
🌈 Barra Superior	Línea degradada animada con efecto shimmer
🎬 Animaciones	Múltiples animaciones suaves y profesionales
🕒 Botones Modernos	Con iconos, texto, efecto ripple y hover
📱 Responsive	Se adapta perfectamente a móviles
🎨 Colores por Tipo	Verde (éxito), Rojo (error), etc.
🎉 Partículas	Emojis flotantes en notificaciones de éxito

10.2 Animaciones


```
1  /* Entrada de la notificación */
2  @keyframes slideUpBounce {
3    0% {
4      transform: translateY(100px) scale(0.8) rotate(-5deg);
5      opacity: 0;
6    }
7    100% {
8      transform: translateY(0) scale(1) rotate(0deg);
9      opacity: 1;
10   }
11 }
12
13 /* Animación del icono */
14 @keyframes iconPop {
15   0% { transform: scale(0) rotate(-180deg); }
16   100% { transform: scale(1) rotate(0deg); }
17 }
18
19 /* Anillo pulsante */
20 @keyframes ringPulse {
21   0%, 100% { transform: scale(1); opacity: 1; }
22   50% { transform: scale(1.1); opacity: 0.5; }
23 }
24
25 /* Efecto shimmer en barra */
26 @keyframes shimmer {
27   0% { background-position: 200% 0; }
28   100% { background-position: -200% 0; }
29 }
```

11. SOLUCIÓN DE PROBLEMAS

Errores Comunes y Soluciones

Error	Causa	Solución	↓
<code>npm: command not found</code>	Node.js no instalado	Instalar Node.js desde nodejs.org	
<code>Access denied for user 'root'</code>	Contraseña MySQL incorrecta	Verificar <code>.env</code> con la contraseña correcta	
<code>Cannot find module 'express'</code>	Dependencias no instaladas	Ejecutar <code>npm install</code> en backend/	
<code>Port 3001 already in use</code>	Puerto ocupado	Cambiar PORT en <code>.env</code> o cerrar el proceso	
<code>CORS error</code>	Configuración de CORS	Verificar <code>app.use(cors())</code> en <code>index.js</code>	
<code>Token expired</code>	Token JWT vencido	Iniciar sesión nuevamente	
<code>404 Not Found</code>	Ruta no existe	Verificar que la ruta está montada en <code>index.js</code>	
<code>ER_DUP_ENTRY</code>	Datos duplicados	Usar valores únicos para registro/correo	
<code>router is not defined</code>	Falta definir router	Agregar <code>const router = express.Router();</code>	
<code>Cannot resolve './Notification'</code>	Archivo no existe	Crear <code>Notification.jsx</code> y <code>Notification.css</code>	